

Documentacion tecnica y guia de defensa - JurisApp

Proyecto: JurisApp

Objetivo: plataforma SaaS para abogados en Argentina, pensada para asistir tareas legales con autenticacion, perfiles profesionales, chats con IA, documentos, custom skills, tareas IA por pasos, carpetas y planes de suscripcion.

Este documento esta pensado para estudiar y defender el proyecto. Resume las carpetas importantes, las capas, los archivos principales y los flujos que conviene explicar.

1. Resumen ejecutivo

JurisApp es una aplicacion para abogados que centraliza trabajo legal asistido por IA. Permite:

- Registrar usuarios, verificar email, iniciar sesion y recuperar contrasena.
- Solicitar verificacion de abogado y aprobar/rechazar solicitudes desde un rol administrador.
- Crear chats legales asociados al usuario.
- Enviar consultas a un servicio de IA.
- Adjuntar documentos al chat y usarlos como contexto.
- Analizar documentos legales y guardar resumen, riesgos, recomendaciones y referencias.
- Crear custom skills, es decir, instrucciones reutilizables para adaptar la IA al estilo o necesidad del abogado.
- Crear tareas IA con plan de trabajo editable, aprobacion humana y ejecucion paso a paso.
- Organizar trabajo en carpetas.
- Manejar planes Free, Pro y Max, con Stripe real o modo simulado en desarrollo.

La parte mas fuerte del proyecto esta en el backend, construido con Clean Architecture en .NET. El frontend incluido en `frontend/` es una base React/Vite. La interfaz de prueba funcional para demostrar todos los flujos esta en `backend/Presentation/wwwroot/test.html`.

2. Stack tecnologico

Backend

- .NET 10.
- ASP.NET Core Web API.
- Entity Framework Core.
- SQLite en desarrollo.
- PostgreSQL previsto para produccion.
- JWT Bearer para autenticacion.
- BCrypt para hash de contrasenas.
- Swagger/OpenAPI para documentar y probar endpoints.
- Claude/Anthropic como servicio de IA, con fallback/mock de desarrollo si no hay API key.
- Stripe para suscripciones pagas, con modo mock en desarrollo.
- OpenXML para leer `.docx`.
- PdfPig para extraer texto de PDF.

Frontend

- React 19.
- Vite.
- ESLint.
- Assets SVG/PNG basicos.

Persistencia y archivos

- Base de datos administrada por EF Core y migraciones.
- Archivos subidos en desarrollo dentro de `backend/Presentation/wwwroot/uploads`.
- La base local de desarrollo se configura como `jurisapp.dev.db` dentro de `backend/Presentation`.

3. Arquitectura general

El backend sigue una arquitectura limpia por capas:

```
```text
Presentation -> Application -> Domain
 | ^
 v |
Infrastructure -----+
```
```

La idea es separar responsabilidades:

- `Domain` define el negocio puro: entidades, enums y reglas internas.
- `Application` define casos de uso, DTOs, contratos e interfaces.
- `Infrastructure` implementa detalles tecnicos: base de datos, repositorios, JWT, archivos, IA, pagos.
- `Presentation` expone la API HTTP y conecta la aplicacion con el mundo externo.

Una forma simple de explicarlo:

> Los controladores no conocen Entity Framework ni Stripe directamente. Llamam servicios de aplicacion. Los servicios dependen de interfaces. Infrastructure implementa esas interfaces. Domain queda independiente y representa el nucleo del negocio.

4. Estructura principal del repositorio

```
```text
JurisApp/
 README.md
 DOCUMENTACION_JURISAPP.md
 backend/
 Solution.slnx
 Domain/
 Application/
 Infrastructure/
 Presentation/
 frontend/
 package.json
 index.html
 src/
 public/
```
```

Carpetas que si importan

- `backend/Domain`: modelo de negocio.
- `backend/Application`: servicios, DTOs, interfaces y reglas de aplicacion.
- `backend/Infrastructure`: implementaciones tecnicas.
- `backend/Presentation`: API, Swagger, configuracion y test HTML.
- `frontend/src`: entrada React/Vite.

Carpetas que no hace falta defender en detalle

- `frontend/node\_modules`: dependencias instaladas de Node.
- `backend/\*\*/bin` y `backend/\*\*/obj`: compilacion generada por .NET.
- `backend/Presentation/wwwroot/uploads`: archivos subidos en pruebas.
- `backend/Infrastructure/Migrations`: importante saber para que esta, pero no estudiar cada linea generada.

5. Capa Domain

Ruta: `backend/Domain`

Es la capa mas interna. No depende de ASP.NET, EF Core, Stripe ni Claude. Contiene las clases que representan el negocio.

Archivos clave

```
Archivo/carpeta	Responsabilidad
`Domain.csproj`	Proyecto .NET de la capa de dominio. Target `net10.0`.
`Common/BaseEntity.cs`	Clase base con `Id`, `CreatedAt`, `UpdatedAt` y metodo `Touch()` para
actualizar fecha.	
`Entities/`	Entidades principales del negocio.
`Enums/`	Estados y tipos usados por las entidades.
```

Entidades principales

```
Entidad	Que representa	Datos importantes
`User`	Usuario de la plataforma.	Nombre, apellido, email, password hash, rol, estado activo,
email verificado, tema visual.		
`LawyerProfile`	Perfil profesional de abogado.	Matricula, colegio, provincia, especialidad,
estado de verificacion.		
`Chat`	Conversacion legal del usuario.	Usuario propietario, titulo, carpeta opcional, mensajes,
documentos, tareas y skills aplicadas.		
`Message`	Mensaje dentro de un chat.	Rol del mensaje, contenido, fecha y skills usadas.
`Document`	Documento adjunto al chat.	Titulo, URL/ruta, chat y carpeta opcional.
`DocumentAnalysis`	Resultado de analisis de documento.	Resumen, riesgos, recomendaciones,
referencias y tipo de analisis.		
`Folder`	Carpeta o expediente del abogado.	Nombre, contexto legal, chats y documentos
asociados.		
`CustomSkill`	Instruccion personalizada para la IA.	Nombre, cuando usarla, instrucciones,
ejemplos, alertas, formato de salida, activo/inactivo.		
`ChatCustomSkill`	Relacion entre un chat y una skill aplicada.	Chat, skill y fecha de
aplicacion.		
`AITask`	Tarea IA estructurada.	Descripcion, estado, plan, resultado, paso actual, pausada/no
pausada.		
`AITaskStep`	Paso individual de una tarea IA.	Orden, titulo, descripcion, estado y resultado.
`Plan`	Plan comercial.	Nombre, tipo Free/Pro/Max, precio, limites JSON, IDs de Stripe.
`Subscription`	Suscripcion de usuario.	Usuario, plan, fecha de inicio/fin, estado e IDs Stripe.
`PasswordResetToken`	Token de recuperacion de password.	Usuario, hash del token, vencimiento,
uso.		
`EmailVerificationToken`	Codigo de verificacion de email.	Usuario, hash del codigo,
vencimiento, uso.		
`Audit`	Auditoria potencial de IA.	Chat, modelo y version de prompt. Esta configurada pero no
es central en los servicios actuales. |
```

Enums importantes

```
Enum	Valores	Para que sirve
`UserRole`	`User`, `Lawyer`, `Admin`	Control de permisos.
`UserTheme`	`Bright`, `Dark`	Preferencia visual del usuario.
`LawyerVerificationStatus`	`NotSubmitted`, `Pending`, `Verified`, `Rejected`	Estado de
solicitud profesional.		
`MessageRole`	`User`, `Assistant`, `System`	Tipo de mensaje en chat.
`DocumentAnalysisType`	`Summary`, `RiskAnalysis`, `ContractReview`, `Custom`	Tipo de analisis
legal.		
`AITaskStatus`	`Pending`, `AwaitingApproval`, `InProgress`, `Completed`, `Failed`, `Cancelled`	
Ciclo de vida de tarea IA.		
`AITaskStepStatus`	`Pending`, `InProgress`, `Completed`, `Failed`, `Skipped`	Estado de cada
```

```
paso. |
| `PlanType` | `Free`, `Pro`, `Max` | Categoria comercial. |
| `SubscriptionStatus` | `Active`, `Cancelled`, `Expired` | Estado de suscripcion. |
```

Reglas de dominio destacables

- `User.VerifyEmail()` marca el email como verificado.
- `User.UpgradeToLawyer()` cambia el rol a abogado cuando un admin aprueba el perfil.
- `LawyerProfile.Verify()` aprueba una solicitud y registra quien la verifico.
- `LawyerProfile.RejectVerification()` rechaza una solicitud y guarda motivo.
- `Chat.ApplySkill()` y `Chat.RemoveSkill()` administran skills activas en un chat.
- `AITask.ApprovePlan()` pasa una tarea a ejecucion y arranca en el paso 1.
- `AITask.Pause()`, `Resume()`, `Cancel()` y `MarkAsCompleted()` modelan el flujo de tareas IA.
- `AITaskStep.MarkAsCompleted()` guarda el resultado de cada paso.
- `Subscription.ActivateFromPayment()` activa una suscripcion paga desde Stripe o mock.

6. Capa Application

Ruta: `backend/Application`

Es la capa de casos de uso. No expone HTTP ni sabe detalles de EF Core. Define que acciones permite el sistema y coordina entidades, validaciones e interfaces.

Carpetas principales

```
Carpeta	Responsabilidad
`Services/`	Implementa los casos de uso del sistema.
`Interfaces/Services`	Contratos que consume Presentation.
`Interfaces/Persistence`	Contratos de repositorios que implementa Infrastructure.
`Interfaces/Auth`	Contratos para JWT, password hashing, usuario actual y email.
`Interfaces/AI`	Contrato del servicio de IA y modelos auxiliares.
`Interfaces/Files`	Contratos para guardar archivos y extraer texto.
`Interfaces/Payments`	Contrato para Stripe/mock payments.
`DTOs/`	Objetos de entrada/salida de la API.
`Mappings/`	Convierte entidades de dominio a DTOs.
`Common/`	Resultado uniforme, errores, paginacion y excepcion de IA.
`Auth/AuthValidators.cs`	Validaciones de email, password, codigos y tokens.
`DependencyInjection.cs`	Registra servicios de aplicacion en DI.
```

Servicios de aplicacion

```
Servicio	Responsabilidad
`AuthService`	Registro, login, verificacion de email, reenvio de codigo, forgot/reset password.
`UserService`	Perfil de usuario, listado/admin update de usuarios, cambio de tema.
`LawyerProfileService`	Solicitud, edicion, aprobacion y rechazo de perfiles de abogado.
`ChatService`	Crear/listar/ver/eliminar chats y enviar mensajes a la IA.
`ChatDocumentContextService`	Construye contexto de documentos para enviar a la IA.
`DocumentService`	Subida de documentos, lectura, analisis con IA y consulta por chat.
`CustomSkillService`	Crear/editar/listar/eliminar skills, activarlas y aplicarlas a chats.
`FolderService`	Crear, editar, listar y eliminar carpetas del abogado.
`AITaskService`	Crear plan IA, editarlo, aprobarlo, ejecutar pasos, pausar, reanudar y cancelar.
`PlanService`	Listar planes, suscribirse a Free, consultar plan actual y activar pagos.
```

Common: Result y Error

El proyecto usa `Result` y `Result<T>` para devolver exito o error sin tirar excepciones por cada validacion. Esto permite que los servicios de aplicacion digan:

- exito con valor (`Result<T>.Success(...)`);
- error de validacion;

- no encontrado;
- no autorizado;
- conflicto;
- error de servicio externo.

Luego Presentation convierte esos errores a HTTP con `ResultExtensions`.

Ejemplo defendible:

> En vez de que cada controlador arme manualmente respuestas HTTP, los servicios devuelven un resultado uniforme y `ResultExtensions` traduce eso a `200`, `400`, `401`, `404`, `409` o `502`.

DTOs

Los DTOs separan el modelo interno de dominio de lo que entra/sale por API. Ejemplos:

- `RegisterRequest`, `LoginRequest`, `AuthResponse`.
- `CreateChatRequest`, `ChatDto`, `MessageDto`.
- `UploadDocumentRequest`, `DocumentDto`, `DocumentAnalysisDto`.
- `CreateCustomSkillRequest`, `CustomSkillDto`.
- `CreateAITaskRequest`, `AITaskDetailDto`, `TaskStepDto`.
- `PlanDto`, `SubscriptionDto`, `CurrentPlanDto`.

Esto evita exponer directamente entidades con propiedades internas como `PasswordHash`.

7. Capa Infrastructure

Ruta: `backend/Infrastructure`

Implementa los detalles tecnicos que Application solo conoce como interfaces.

Carpetas principales

```
Carpeta/archivo	Responsabilidad
`DependencyInjection.cs`	Registra base de datos, repositorios, auth, IA, archivos y pagos.
`Persistence/AppDbContext.cs`	DbContext EF Core y DbSet.
`Persistence/Repositories/`	Implementaciones concretas de repositorios.
`Persistence/Configurations/`	Configuracion EF Core por entidad.
`Persistence/Migrations/`	Historial de cambios de base de datos.
`Persistence/DevDataSeeder.cs`	Seed de planes y admin en desarrollo.
`Auth/`	Hash de passwords, JWT, usuario actual y email por logs.
`AI/`	Integracion Claude, mock/fallback y parser de planes.
`Files/`	Guardado local y extraccion de texto de documentos.
`Payments/`	Integracion Stripe y mock de pagos.
```

Persistencia

`AppDbContext.cs` declara estas tablas principales:

- `Users`
- `PasswordResetTokens`
- `EmailVerificationTokens`
- `LawyerProfiles`
- `Chats`
- `Messages`
- `Audits`
- `Documents`
- `DocumentAnalyses`
- `Folders`
- `CustomSkills`
- `ChatCustomSkills`
- `AITasks`

- `AITaskSteps`
- `Plans`
- `Subscriptions`

Las configuraciones EF Core definen:

- Claves primarias.
- Longitudes maximas.
- Relaciones entre entidades.
- Cascadas o `SetNull` al borrar.
- Indices unicos como email de usuario y relacion chat-skill.
- Conversion de enums a string para guardar estados de forma legible.

Base de datos por entorno

En `Infrastructure/DependencyInjection.cs`:

- Si el entorno es Development y `Database:Provider` es `Sqlite`, usa SQLite.
- Si no, usa PostgreSQL con Npgsql.

Esto permite desarrollo local simple y produccion mas robusta.

Auth

| Archivo | Funcion |
|-------------------------|--|
| `PasswordHasher.cs` | Usa BCrypt para hashear y verificar passwords. |
| `JwtTokenGenerator.cs` | Genera tokens JWT con claims de usuario y rol. |
| `CurrentUserService.cs` | Lee el usuario actual desde los claims HTTP. |
| `LoggingEmailSender.cs` | En desarrollo escribe codigos/links en logs en vez de enviar email real. |

IA

| Archivo | Funcion |
|---------------------|--|
| `AIService.cs` | Implementa chat, analisis de documentos, plan estructurado y ejecucion de pasos usando Claude. |
| `ClaudeOptions.cs` | Opciones de configuracion: API key, modelo, base URL, max tokens. |
| `MockAIService.cs` | Implementacion simulada de IA. |
| `TaskPlanParser.cs` | Convierte respuesta JSON de IA en plan estructurado y tiene plan mock. |

Punto importante: `AIService` tiene fallback. Si no esta habilitado el modo live o falta API key, responde con textos simulados. Eso permite demostrar la aplicacion sin depender siempre de una API externa.

Archivos y documentos

| Archivo | Funcion |
|------------------------------|---|
| `LocalFileStorageService.cs` | Guarda archivos localmente y permite abrirlos. |
| `DocumentTextExtractor.cs` | Extrae texto de `.txt`, `.md`, `.csv`, `.json`, `.xml`, `.html`, `.pdf`, `.docx`, `.rtf`. |

`ChatDocumentContextService` limita el contexto enviado a la IA:

- maximo 12.000 caracteres por documento;
- maximo 30.000 caracteres en total.

Esto evita enviar prompts excesivos.

Pagos

| Archivo | Funcion |
|---------|---------|
| | |

```
`StripePaymentService.cs`	Crea sesiones de checkout y procesa webhooks reales.
`MockPaymentService.cs`	Simula pagos en desarrollo.
`StripeOptions.cs`	Configuración de Stripe.
```

En desarrollo existe endpoint de simulación: `POST /api/billing/simulate-purchase`.

8. Capa Presentation

Ruta: `backend/Presentation`

Es la API HTTP. Contiene el punto de entrada, controladores, configuración y herramientas de prueba.

Archivos clave

```
Archivo/carpeta	Responsabilidad
`Program.cs`	Configura DI, controllers, JSON enums, Swagger, JWT, CORS, migraciones, seed,
archivos estáticos y pipeline HTTP.	
`Controllers/`	Endpoints REST de la API.
`Extensions/ResultExtensions.cs`	Convierte `Result` de Application en respuestas HTTP.
`appsettings.json`	Configuración base.
`appsettings.Development.json`	Configuración local: SQLite, JWT dev, Claude, Stripe mock, CORS.
`Properties/launchSettings.json`	Puertos y perfiles HTTP/HTTPS.
`wwwroot/test.html`	Interfaz manual para probar los flujos sin frontend completo.
`Presentation.http`	Archivo de pruebas HTTP.
`Presentation.csproj`	Proyecto Web API.
```

Program.cs explicado

`Program.cs` hace lo siguiente:

1. Crea el builder de ASP.NET.
2. Registra Application e Infrastructure.
3. Agrega controllers y serialización de enums como strings.
4. Configura Swagger con seguridad Bearer.
5. Configura JWT Bearer.
6. Configura autorización.
7. Configura CORS para el frontend y URLs locales.
8. Inicializa la app.
9. En Development aplica migraciones y seed automáticamente.
10. Habilita Swagger en Development.
11. Sirve archivos estáticos.
12. Usa CORS, autenticación y autorización.
13. Habilita buffering para webhook de Stripe.
14. Mapea controllers.

ResultExtensions.cs

Traduce errores del dominio/aplicación a HTTP:

```
Error	HTTP
Éxito sin valor	`200 OK`
Éxito con valor	`200 OK` + JSON
Creación	`201 Created`
`NotFound`	`404 Not Found`
`Unauthorized`	`401 Unauthorized`
`Conflict`	`409 Conflict`
`ExternalService`	`502 Bad Gateway`
Otros	`400 Bad Request`
```

9. Controladores y endpoints

AuthController - `api/auth`

Endpoints publicos para identidad:

| Metodo | Ruta | Funcion |
|--------|---------------------------------|--|
| POST | `/api/auth/register` | Crea usuario y envia codigo de verificacion. |
| POST | `/api/auth/verify-email` | Verifica email con codigo y devuelve JWT. |
| POST | `/api/auth/resent-verification` | Reenvia codigo de verificacion. |
| POST | `/api/auth/login` | Inicia sesion y devuelve JWT. |
| POST | `/api/auth/forgot-password` | Genera link/token de recuperacion. |
| POST | `/api/auth/reset-password` | Cambia password usando token valido. |

UsersController - `api/users`

Requiere JWT. Algunas rutas requieren Admin.

| Metodo | Ruta | Permiso | Funcion |
|--------|-------------------|---------------------|--------------------------------|
| GET | `/api/users/me` | Usuario autenticado | Perfil actual. |
| PUT | `/api/users/me` | Usuario autenticado | Edita nombre, apellido y tema. |
| GET | `/api/users` | Admin | Lista usuarios. |
| GET | `/api/users/{id}` | Admin | Detalle de usuario. |
| PUT | `/api/users/{id}` | Admin | Cambia rol o estado activo. |

LawyerProfilesController - `api/lawyer-profiles`

Maneja solicitudes de verificacion profesional.

| Metodo | Ruta | Permiso | Funcion |
|--------|--|---------------------|---------------------------------------|
| POST | `/api/lawyer-profiles` | Usuario autenticado | Envia solicitud de abogado. |
| GET | `/api/lawyer-profiles/me` | Usuario autenticado | Ver solicitud/perfil propio. |
| PUT | `/api/lawyer-profiles/me` | Usuario autenticado | Editar solicitud si corresponde. |
| GET | `/api/lawyer-profiles/requests` | Admin | Listar solicitudes. |
| GET | `/api/lawyer-profiles/requests/{id}` | Admin | Ver detalle. |
| POST | `/api/lawyer-profiles/requests/{id}/approve` | Admin | Aprobar solicitud. |
| POST | `/api/lawyer-profiles/requests/{id}/reject` | Admin | Rechazar solicitud. |
| POST | `/api/lawyer-profiles/verify` | Admin | Endpoint alternativo de verificacion. |
| POST | `/api/lawyer-profiles/reject` | Admin | Endpoint alternativo de rechazo. |

ChatsController - `api/chats`

Requiere JWT.

| Metodo | Ruta | Funcion |
|--------|----------------------------|--|
| POST | `/api/chats` | Crear chat. |
| GET | `/api/chats` | Listar chats del usuario. |
| GET | `/api/chats/{id}` | Cargar chat con mensajes. |
| POST | `/api/chats/{id}/messages` | Enviar mensaje y obtener respuesta IA. |
| DELETE | `/api/chats/{id}` | Eliminar chat. |

DocumentsController - `api/documents`

Requiere JWT.

| Metodo | Ruta | Funcion |
|--------|--------------------------------|-------------------------------|
| POST | `/api/documents/upload` | Subir documento a un chat. |
| GET | `/api/documents/{id}` | Obtener documento por ID. |
| GET | `/api/documents/chat/{chatId}` | Listar documentos de un chat. |
| POST | `/api/documents/analyze` | Analizar documento con IA. |

CustomSkillsController - `api/custom-skills`

Requiere rol `Lawyer` o `Admin`.

| Metodo | Ruta | Funcion |
|--------|---|------------------------|
| POST | `/api/custom-skills` | Crear skill. |
| PUT | `/api/custom-skills/{id}` | Editar skill. |
| GET | `/api/custom-skills/me` | Listar skills propias. |
| GET | `/api/custom-skills/lawyer-profile/{lawyerProfileId}` | Listar por perfil. |
| POST | `/api/custom-skills/{id}/activate` | Activar skill. |
| POST | `/api/custom-skills/{id}/deactivate` | Desactivar skill. |
| POST | `/api/custom-skills/apply` | Aplicar skill a chat. |
| POST | `/api/custom-skills/remove` | Quitar skill del chat. |
| DELETE | `/api/custom-skills/{id}` | Eliminar skill. |

FoldersController - `api/folders`

Requiere rol `Lawyer` o `Admin`.

| Metodo | Ruta | Funcion |
|--------|---------------------|------------------------------|
| POST | `/api/folders` | Crear carpeta/expediente. |
| PUT | `/api/folders/{id}` | Editar carpeta. |
| GET | `/api/folders` | Listar carpetas del abogado. |
| DELETE | `/api/folders/{id}` | Eliminar carpeta. |

AITasksController - `api/ai-tasks`

Requiere JWT.

| Metodo | Ruta | Funcion |
|--------|-----------------------------------|-------------------------------------|
| POST | `/api/ai-tasks` | Crear tarea IA y plan estructurado. |
| GET | `/api/ai-tasks/{id}` | Ver tarea por ID. |
| PUT | `/api/ai-tasks/{id}/plan` | Editar pasos antes de aprobar. |
| POST | `/api/ai-tasks/{id}/approve` | Aprobar plan y ejecutar. |
| POST | `/api/ai-tasks/{id}/execute-next` | Ejecutar siguiente paso. |
| POST | `/api/ai-tasks/{id}/pause` | Pausar tarea. |
| POST | `/api/ai-tasks/{id}/resume` | Reanudar tarea. |
| POST | `/api/ai-tasks/{id}/cancel` | Cancelar tarea. |
| GET | `/api/ai-tasks/chat/{chatId}` | Listar tareas de un chat. |

PlansController - `api/plans`

| Metodo | Ruta | Permiso | Funcion |
|--------|----------------------------------|---------|---------------------------|
| GET | `/api/plans` | Publico | Lista planes. |
| POST | `/api/plans/{planId}/subscribe` | JWT | Suscripcion al plan Free. |
| GET | `/api/plans/subscription/active` | JWT | Ver suscripcion activa. |
| GET | `/api/plans/current` | JWT | Ver plan actual. |

BillingController - `api/billing`

| Metodo | Ruta | Permiso | Funcion |
|--------|--|--------------------------|-----------------------------------|
| POST | `/api/billing/create-checkout-session` | JWT | Crea checkout Stripe. |
| POST | `/api/billing/simulate-purchase` | JWT + Development | Activa plan pago sin Stripe real. |
| POST | `/api/billing/webhook` | Publico con firma Stripe | Procesa pago completado. |

10. Flujos funcionales principales

Flujo 1: Registro y login

```text

Usuario -> POST /api/auth/register  
 -> AuthService valida email/password  
 -> UserRepository guarda usuario con password hasheado  
 -> EmailVerificationTokenRepository guarda codigo hasheado  
 -> LoggingEmailSender muestra codigo en logs

Usuario -> POST /api/auth/verify-email  
 -> se valida codigo  
 -> User.VerifyEmail()  
 -> se devuelve JWT

Usuario -> POST /api/auth/login  
 -> verifica password BCrypt  
 -> verifica usuario activo y email verificado  
 -> devuelve JWT

```

Defensa:

> Nunca se guarda la password en texto plano. Se guarda un hash BCrypt. Los codigos de verificacion y tokens de recuperacion tambien se guardan hasheados.

Flujo 2: Verificacion de abogado

```text

Usuario autenticado -> envia matricula, colegio, provincia, especialidad  
 Application -> crea LawyerProfile en estado Pending  
 Admin -> lista solicitudes  
 Admin -> aprueba o rechaza  
 Si aprueba -> LawyerProfile pasa a Verified y User pasa a rol Lawyer

```

Esto separa usuarios comunes de abogados verificados. Permite restringir features como carpetas y custom skills.

Flujo 3: Chat con IA

```text

Usuario -> crea chat  
 Usuario -> envia mensaje  
 ChatService -> verifica que el chat pertenece al usuario  
 ChatService -> carga skills activas del chat  
 ChatDocumentContextService -> carga documentos del chat como contexto  
 AIService -> envia prompt a Claude o devuelve fallback  
 MessageRepository -> guarda mensaje del usuario y respuesta del asistente

```

Valor del flujo:

- El chat tiene memoria por mensajes previos.
- Las custom skills modifican la respuesta.
- Los documentos adjuntos se agregan como contexto.

Flujo 4: Documentos y analisis

```text

Usuario -> sube archivo a un chat  
 DocumentService -> valida propiedad del chat  
 LocalFileStorageService -> guarda archivo  
 DocumentRepository -> registra documento

Usuario -> pide analizar documento

```
DocumentService -> abre archivo
DocumentTextExtractor -> extrae texto
AIService -> analiza segun tipo elegido
DocumentAnalysisRepository -> guarda resultado
```
```

Formatos soportados:

- PDF
- DOCX
- RTF como texto
- texto plano y variantes: TXT, MD, CSV, JSON, XML, HTML, LOG

Flujo 5: Custom skills

```
```text
Abogado/Admin -> crea skill con instrucciones
Skill -> queda activa por defecto
Usuario -> aplica skill a un chat
ChatService/AIService -> incluye instrucciones de skills activas en el prompt
```
```

Ejemplo:

Una skill "Revision de contrato" puede indicar que la IA revise clausulas de rescision, penalidades, riesgos y formato de salida.

Flujo 6: Tareas IA por pasos

```
```text
Usuario -> describe encargo legal en Modo Tarea IA
AITaskService -> pide a IA un plan estructurado JSON
Sistema -> guarda AITask y AITaskStep
Usuario -> puede editar titulos/descripciones de pasos
Usuario -> aprueba el plan
Sistema -> ejecuta paso por paso
Cada paso -> genera resultado y mensaje en el chat
Usuario -> puede pausar, reanudar o cancelar
```
```

Este flujo es importante para la defensa porque muestra "human in the loop":

- La IA propone.
- El abogado revisa/edita.
- Recien despues se ejecuta.
- Cada paso queda persistido.

Flujo 7: Planes y billing

```
```text
Usuario -> consulta planes
Plan Free -> se puede activar directo
Plan Pro/Max -> requiere Stripe checkout
Development -> se puede simular compra
Webhook Stripe -> confirma pago y activa suscripcion
```
```

En desarrollo:

- `Stripe:UseMock` esta en `true`.
- `simulate-purchase` permite probar Pro/Max sin claves reales.

Flujo 8: Admin usuarios

```
```text
```

```
Admin -> lista usuarios
Admin -> cambia rol o estado activo
Sistema -> impide que admin se modifique a si mismo
```
```

Sirve para control interno y pruebas de permisos.

11. Seguridad y permisos

JWT

El usuario autenticado recibe un token JWT. Luego los endpoints protegidos usan `[Authorize]`.

Roles

Roles definidos:

- `User`: usuario comun.
- `Lawyer`: abogado verificado.
- `Admin`: administrador.

Endpoints con rol especial:

- Custom skills: `Lawyer` o `Admin`.
- Folders: `Lawyer` o `Admin`.
- Gestion de usuarios: `Admin`.
- Gestion de solicitudes de abogado: `Admin`.

Propiedad de recursos

Los servicios validan que:

- Un chat pertenezca al usuario antes de leerlo o enviar mensajes.
- Un documento pertenezca a un chat del usuario.
- Una carpeta pertenezca al perfil de abogado del usuario.
- Una custom skill pertenezca al perfil de abogado del usuario.
- Una tarea IA pertenezca a un chat del usuario.

Passwords y tokens

- Passwords con BCrypt.
- Tokens/codigos de verificacion hasheados.
- Email no verificado no puede iniciar sesion.
- Usuario inactivo no puede iniciar sesion.

12. Configuracion y ejecucion

Backend

Desde `backend/Presentation`:

```
```bash
dotnet restore
dotnet build
dotnet run
```
```

URLs por defecto:

- API HTTP: `http://localhost:5248`
- API HTTPS: `https://localhost:7212`

- Swagger: `http://localhost:5248/swagger`
- Interfaz de prueba: `http://localhost:5248/test.html`

Development

Archivo: `backend/Presentation/appsettings.Development.json`

Incluye:

- SQLite: `Data Source=jurisapp.dev.db`
- `Database:Provider = Sqlite`
- JWT secret de desarrollo.
- Claude habilitado por configuracion, pero requiere API key para modo live.
- Stripe mock activo.
- CORS para `localhost:5173`, `localhost:5248`, `localhost:7212`.
- Storage local en `wwwroot/uploads`.

En Development, `Program.cs` hace:

- `db.Database.MigrateAsync()`
- `DevDataSeeder.SeedAsync(...)`

Es decir, aplica migraciones y crea datos base automaticamente.

Seed de desarrollo

Se crean:

- Planes `Free`, `Pro`, `Max`.
- Usuario admin: `admin@jurisapp.local` / `Admin123!`.

Frontend

Desde `frontend`:

```
```bash
npm install
npm run dev
```
```

El frontend configurado es una base React/Vite. Para demostracion completa del backend, usar `http://localhost:5248/test.html`.

13. Frontend

Ruta: `frontend`

Archivos importantes

| Archivo | Funcion |
|--|---|
| `package.json` | Scripts y dependencias React/Vite. |
| `index.html` | HTML raiz donde se monta React. |
| `src/main.jsx` | Punto de entrada React. Renderiza ` <app >`.<="" td=""> </app> |
| `src/App.jsx` | Componente principal actual. |
| `src/App.css` e `src/index.css` | Estilos. |
| `public/icons.svg`, `public/favicon.svg` | Iconos publicos. |
| `src/assets/` | Assets usados por el componente. |

Como explicarlo

El frontend actual en `frontend/src` es una base Vite/React. El proyecto incluye ademas una interfaz HTML de prueba mucho mas completa en `backend/Presentation/wwwroot/test.html`, que permite probar:

- registro;
- verificacion de email;
- login;
- usuarios/admin;
- solicitud de abogado;
- chats;
- documentos;
- custom skills;
- tareas IA;
- planes y billing.

Para la entrega, si preguntan por la demostracion funcional, conviene usar `test.html` junto con Swagger.

14. Base de datos y modelo relacional

Relaciones principales:

```text

User 1 --- 0..1 LawyerProfile

User 1 --- N Chat

User 1 --- N Subscription

LawyerProfile 1 --- N Folder

LawyerProfile 1 --- N CustomSkill

Chat 1 --- N Message

Chat 1 --- N Document

Chat 1 --- N AITask

Chat N --- N CustomSkill via ChatCustomSkill

Chat N --- 0..1 Folder

Document 1 --- 0..1 DocumentAnalysis

Document N --- 0..1 Folder

AITask 1 --- N AITaskStep

Plan 1 --- N Subscription

```

Puntos defendibles:

- La relacion `ChatCustomSkill` evita duplicar skills y permite aplicarlas/desaplicarlas por chat.
- `DocumentAnalysis` separado de `Document` permite guardar resultado estructurado del analisis.
- `AITask` y `AITaskStep` separados permiten auditar el avance paso a paso.
- `Folder` organiza chats y documentos bajo un perfil de abogado.
- `Subscription` separado de `Plan` permite historial/cambios de suscripcion.

15. Archivos clave para estudiar

Backend - entrada y configuracion

| Archivo | Por que importa |

|---|---|

| `backend/Presentation/Program.cs` | Muestra como se arma toda la app. |

| `backend/Presentation/appsettings.Development.json` | Configuracion local real. |

| `backend/Presentation/Extensions/ResultExtensions.cs` | Traduccion de errores a HTTP. |

| `backend/Infrastructure/DependencyInjection.cs` | Registro de implementaciones concretas. |

| `backend/Application/DependencyInjection.cs` | Registro de servicios de negocio. |

Backend - negocio

Archivo	Por que importa
`backend/Application/Services/AuthService.cs`	Flujo de identidad completo.
`backend/Application/Services/ChatService.cs`	Chat con IA y skills.
`backend/Application/Services/DocumentService.cs`	Subida y analisis de documentos.
`backend/Application/Services/AITaskService.cs`	Planes IA editables y ejecucion paso a paso.
`backend/Application/Services/LawyerProfileService.cs`	Verificacion profesional.
`backend/Application/Services/CustomSkillService.cs`	Skills personalizadas.
`backend/Application/Services/PlanService.cs`	Plan actual y suscripciones.

Backend - dominio

Archivo	Por que importa
`backend/Domain/Entities/User.cs`	Usuario, roles, email verificado, estado activo.
`backend/Domain/Entities/LawyerProfile.cs`	Estado profesional del abogado.
`backend/Domain/Entities/Chat.cs`	Conversaciones y skills aplicadas.
`backend/Domain/Entities/Document.cs`	Documentos adjuntos.
`backend/Domain/Entities/AITask.cs`	Ciclo de vida de tareas IA.
`backend/Domain/Entities/AITaskStep.cs`	Estado y resultado de cada paso.
`backend/Domain/Entities/CustomSkill.cs`	Instrucciones personalizadas para IA.
`backend/Domain/Entities/Subscription.cs`	Estado de suscripcion.

Backend - infraestructura

Archivo	Por que importa
`backend/Infrastructure/Persistence/AppDbContext.cs`	Tablas principales.
`backend/Infrastructure/Persistence/Configurations/*.cs`	Relaciones y restricciones EF Core.
`backend/Infrastructure/AI/AIService.cs`	Como se arma el prompt y se llama a Claude.
`backend/Infrastructure/AI/TaskPlanParser.cs`	Parsing de planes IA.
`backend/Infrastructure/Files/DocumentTextExtractor.cs`	Lectura de PDF/DOCX/texto.
`backend/Infrastructure/Payments/StripePaymentService.cs`	Checkout y webhook real.
`backend/Infrastructure/Payments/MockPaymentService.cs`	Simulacion local de pagos.

Presentation - API

Archivo	Por que importa
`backend/Presentation/Controllers/AuthController.cs`	Endpoints de auth.
`backend/Presentation/Controllers/ChatsController.cs`	Endpoints de chat.
`backend/Presentation/Controllers/DocumentsController.cs`	Endpoints de documentos.
`backend/Presentation/Controllers/AITasksController.cs`	Endpoints de tareas IA.
`backend/Presentation/Controllers/BillingController.cs`	Endpoints de Stripe/mock.
`backend/Presentation/wwwroot/test.html`	Interfaz de demostracion.

16. Guion corto para defender la arquitectura

Pueden decir algo asi:

> JurisApp esta construido con Clean Architecture. En el centro esta `Domain`, donde viven las entidades de negocio como usuarios, perfiles de abogado, chats, documentos, tareas IA, planes y suscripciones. Encima esta `Application`, que contiene los casos de uso: registro, login, verificacion profesional, chats, documentos, custom skills, tareas IA y billing. La capa `Infrastructure` implementa detalles externos como EF Core, SQLite/PostgreSQL, JWT, BCrypt, Claude, Stripe y almacenamiento local. Por ultimo, `Presentation` expone la API REST con controladores, Swagger, autenticacion JWT y una interfaz HTML de prueba.

> Esta separacion permite que el negocio no dependa de la tecnologia. Por ejemplo, `ChatService` no sabe como se implementa Claude o la base de datos; solo usa interfaces. Eso facilita cambiar infraestructura sin reescribir los casos de uso.

17. Guion corto para demo funcional

1. Levantar backend desde `backend/Presentation` con `dotnet run`.
2. Abrir `http://localhost:5248/swagger` para mostrar endpoints.
3. Abrir `http://localhost:5248/test.html`.
4. Registrar usuario.
5. Copiar codigo de verificacion desde logs y verificar email.
6. Login y revisar token.
7. Crear solicitud de abogado.
8. Entrar con admin `admin@jurisapp.local / Admin123!`.
9. Aprobar solicitud.
10. Volver al usuario abogado.
11. Crear chat.
12. Crear/aplicar custom skill.
13. Adjuntar documento.
14. Enviar mensaje normal o generar tarea IA.
15. Editar/aprobar plan IA y mostrar ejecucion por pasos.
16. Listar planes y simular compra Pro/Max si hace falta.

18. Preguntas probables y respuestas

Por que usaron Clean Architecture?

Para separar negocio de infraestructura. Las reglas y casos de uso no quedan mezclados con controladores, base de datos o servicios externos. Eso mejora mantenibilidad, testing y escalabilidad.

Donde esta la logica de negocio?

Principalmente en `backend/Application/Services` y en metodos de entidades de `backend/Domain/Entities`. Los controladores solo reciben HTTP y delegan.

Como protegen los endpoints?

Con JWT Bearer y atributos `[Authorize]`. Algunos endpoints ademas piden roles, por ejemplo `Admin`, `Lawyer` o `Admin/Lawyer`.

Como evitan que un usuario vea datos de otro?

Los servicios validan propiedad: chat por `UserId`, documentos por chat del usuario, carpetas por `LawyerProfileId`, tareas por chat del usuario y skills por perfil de abogado.

Como se conecta la IA?

La interfaz `IAIService` esta en Application. La implementacion `AIService` en Infrastructure arma prompts y llama a Claude. Si no hay modo live/API key, puede devolver respuestas simuladas para desarrollo.

Que aportan las custom skills?

Permiten que un abogado defina instrucciones reutilizables. Al aplicarlas a un chat, la IA recibe esas instrucciones como parte del prompt.

Por que las tareas IA tienen aprobacion?

Porque el abogado debe mantener control profesional. La IA propone un plan, el usuario puede editarlo y recien despues se ejecuta. Es un enfoque human-in-the-loop.

Como se manejan documentos?

Se suben a almacenamiento local, se registran en base de datos y se extrae texto con `DocumentTextExtractor`. Ese texto puede analizarse o usarse como contexto del chat.

Como se manejan pagos?

Los planes estan en base de datos. Free se activa directo. Pro/Max usan Stripe Checkout. En desarrollo hay un modo mock para simular compra sin Stripe real.

Que pasa si Claude falla?

Los errores de IA se encapsulan como `AIServiceException` y se devuelven como `ExternalService`, traducido a HTTP `502`. En modo sin API key hay fallback de desarrollo.

Donde estan las migraciones?

En `backend/Infrastructure/Migrations`. EF Core usa esas migraciones para crear/actualizar la base. En desarrollo se aplican automaticamente al iniciar.

19. Puntos fuertes para mencionar

- Arquitectura limpia y separacion por capas.
- Uso de interfaces para desacoplar casos de uso de implementaciones.
- Seguridad con JWT, roles, BCrypt y validaciones de propiedad.
- Verificacion profesional antes de habilitar funciones de abogado.
- Integracion de IA con contexto de mensajes, documentos y custom skills.
- Tareas IA con plan editable y aprobacion humana.
- Analisis de documentos con extraccion de texto de PDF/DOCX.
- Suscripciones con Stripe y modo mock para desarrollo.
- Swagger y `test.html` facilitan demostracion.
- Seed automatico de planes y admin en desarrollo.

20. Limitaciones o pendientes que se pueden reconocer

Conviene decirlo de forma positiva, como evolucion futura:

- El frontend React principal es una base inicial; la demostracion completa esta en `test.html`.
- En produccion se deben configurar secretos reales: JWT, connection string PostgreSQL, Claude API key y Stripe keys.
- El envio de email real puede reemplazar `LoggingEmailSender`.
- La entidad `Audit` esta preparada/configurada, pero podria integrarse mas profundamente para trazabilidad de IA.
- `PagedResult<T>` existe como base para paginacion futura.

Esto no invalida el proyecto: muestra que ya hay arquitectura preparada para evolucionar.

21. Mapa mental final

```
```text
JurisApp
|
|-- Usuarios
| |-- Registro
| |-- Verificacion de email
| |-- Login JWT
| |-- Roles: User, Lawyer, Admin
|
|-- Abogados
| |-- Solicitud profesional
| |-- Aprobacion admin
```

```
| | -- Carpetas
| | -- Custom Skills
|
|-- Trabajo legal
| | -- Chats
| | -- Mensajes IA
| | -- Documentos
| | -- Analisis documental
| | -- Tareas IA por pasos
|
|-- Monetizacion
| | -- Planes Free/Pro/Max
| | -- Stripe checkout
| | -- Mock de compra en desarrollo
|
|-- Tecnica
| | -- Clean Architecture
| | -- EF Core
| | -- JWT
| | -- Claude
| | -- Swagger
...

```

---

## ## 22. Resumen de una frase

JurisApp es una API SaaS para abogados, construida con Clean Architecture, que combina gestion de usuarios y verificacion profesional con herramientas legales asistidas por IA: chats, documentos, skills personalizadas, tareas planificadas y suscripciones.